Министерство науки и образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №8
по курсу «Логика и основы
алгоритмизации в инженерных задачах»
на тему «Обход графа в ширину»

Выполнил:

студент группы 25ВВВ1
Цеплов Е.В.
Разживалов Н.А.

Приняли:
к.т.н., доцент Юрова О.В.,
к.т.н., доцент Деев М.В.

Пенза 2026

Название

Обход графа в ширину

Цель работы

Изучить обход графа в ширину

Лабораторное задание

1. 1.Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G. Выведите матрицу на экран.
   2.Для сгенерированного графа осуществите процедуру обхода в ширину, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс queue из стандартной библиотеки C++.
   3.Реализуйте процедуру обхода в ширину для графа, представленного списками смежности.
2. 1 Для матричной формы представления графов реализуйте алгоритм обхода в ширину с использованием очереди, построенной на основе структуры данных «список», самостоятельно созданной в лабораторной работе № 3.
   2. Оцените время работы двух реализаций алгоритмов обхода в ширину (использующего стандартный класс queue и использующего очередь, реализованную самостоятельно) для графов разных порядков.

Ход работы

Задание 1:

1. Алгоритм Создание Графа

АЛГОРИТМ СозданиеГрафа(n, M)

Начало

// 1. Инициализация матрицы нулями

для i от 1 до n:

для j от 1 до n:

G[i][j] ← 0

// 2. Генерация рёбер

// Рассматриваем только пары i < j, чтобы не дублировать рёбра

для i от 1 до n:

```
        для j от i + 1 до n:
            если random(0..99) < r:
                G[i][j] ← 1
                G[j][i] ← 1    // граф неориентированный

    // 3. Вывод матрицы на экран
    вывести "    "
    для j от 1 до n:
        вывести j с шириной 3
    вывести перевод строки
    для i от 1 до n:
        вывести i с шириной 3
        для j от 1 до n:
            вывести G[i][j] с шириной 3
        вывести перевод строки
Конец
```

2. Алгоритм BFS

```
Начало
    queue Q                 // std::queue<int>
    Q.push(s)
    NUM[s] ← 1              // вершина посещена
    пока Q не пуста:
        v ← Q.front()
        Q.pop()
        вывести v
```

для u от 1 до n:

если G[v][u] = 1 и NUM[u] = 0:

Q.push(u)

NUM[u] ← 1

Конец

3. Алгоритм через список

Начало

queue Q // std::queue<int>

Q.push(s)

NUM[s] ← 1 // вершина s посещена

пока Q не пуста:

v ← Q.front()

Q.pop()

вывести v

для k от 1 до deg[v]:

u ← M[v][k]

если NUM[u] = 0:

Q.push(u)

NUM[u] ← 1

Конец

Задание 2:

1. Алгоритм BFS через собственную очередь реализованную как структура.

Начало

Q : ListQueue

InitQueue(Q)

```
    PushQueue(Q, s)
    NUM[s] ← 1
    пока не IsEmpty(Q):
        v ← PopQueue(Q)
        вывести v
        для u от 1 до n:
            если G[v][u] = 1 и NUM[u] = 0:
                PushQueue(Q, u)
                NUM[u] ← 1
Конец
```

2.

Тип/n	1000	1500	2000	2500	3000
Queue C++	0.095	0.157	0.207	0.275	0.368
Queue C++ Список	0.101	0.160	0.200	0.270	0.373
Queue	0.092	0.144	0.200	0.255	0.368
Queue Список	0.093	0.146	0.191	0.279	0.372

Вывод: обе реализации BFS имеют одинаковую асимптотику $O(n^2)$, так как работают по матрице смежности. Для графов порядка 1000–3000 разница во времени практически неразличима и находится в пределах погрешности измерения. std::queue обычно работает немного быстрее и стабильнее, а самодельная ListQueue даёт дополнительные накладные расходы на malloc/free.

Результат работы программы

Задание 1:

1. Генерация графа его матрицы и списки.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью
5 - Сравнение скорости
0 - Выход
Ваш выбор: 1

Размер: 4
Вероятность ребра(1-100): 78
      1  2  3  4
  1   0  1  0  0
  2   1  0  0  1
  3   0  0  0  1
  4   0  1  1  0
1. 2
2. 1 4
3. 4
4. 2 3
```

2. Процедура обхода в ширину по матрице.

```
Вероятность ребра(1-100): 78
      1  2  3  4
  1   0  1  0  0
  2   1  0  0  1
  3   0  0  0  1
  4   0  1  1  0
1. 2
2. 1 4
3. 4
4. 2 3

Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 2
1 2 4 3
```

3. Процедура обхода в ширину по спискам смежности.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 3
1 2 4 3
```

Задание 2:

1. Процедура обхода в ширину с помощью собственной очереди сделанной через структуру.

```
Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Сравнение скорости
0 - Выход
Ваш выбор: 4
1 2 4 3
```

```
     1  2  3  4  5
  1   0  1  0  1  1
  2   1  0  0  1  1
  3   0  0  0  1  1
  4   1  1  1  0  1
  5   1  1  1  1  0
1. 2 4 5
2. 1 4 5
3. 4 5
4. 1 2 3 5
5. 1 2 3 4

Выберите вариант работы:
1 - Генерировать граф
2 - Обход в ширину по матрице
3 - Обход в ширину по спискам смежности
4 - Обход в ширину по матрице с помощью очереди через список
5 - Обход в ширину по спискам с помощью очереди через список
6 - Сравнение скорости
0 - Выход
Ваш выбор: 5
1 2 3 4 5
```

2. Сравнение скорости времени работы.

```
Замер номер 1 на 1000
Queue 0,095s:
QueueSP 0,101s:
ListQueue 0,092s:
ListQueueSP 0,093s:
Замер номер 2 на 1500
Queue 0,157s:
QueueSP 0,160s:
ListQueue 0,144s:
ListQueueSP 0,146s:
Замер номер 3 на 2000
Queue 0,207s:
QueueSP 0,200s:
ListQueue 0,200s:
ListQueueSP 0,191s:
Замер номер 4 на 2500
Queue 0,275s:
QueueSP 0,270s:
ListQueue 0,255s:
ListQueueSP 0,279s:
Замер номер 5 на 3000
Queue 0,368s:
QueueSP 0,373s:
ListQueue 0,368s:
ListQueueSP 0,372s:
```

Вывод

В ходе лабораторной работы были изучены матрица и списки смежности графа, реализованы генерация и BFS тремя способами. Обе матричные реализации имеют сложность $O(n^2)$, для $n = 100–300$ разница во времени незначительна; std::queue немного быстрее, списки смежности дают $O(n + m)$. Получены навыки работы с графами и структурами данных на C/C++.